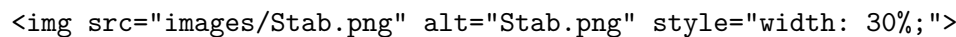


Notebook_StabFEM_backup

May 16, 2025

1 Der Dehnstab

In diesem Notebook wird die FEM-Implementierung für den Dehnstab erläutert und erprobt.

The image shows a diagram of a bar element, likely representing a finite element in a mesh. It is a horizontal line segment with nodes at each end.

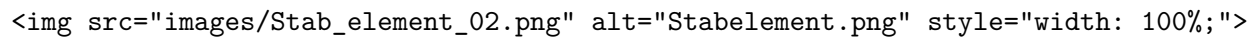
Die Differentialgleichung des Dehnstabes lautet:

$$EAu'' = -n(x)A,$$

Für diese Differentialgleichung haben wir in der Vorlesung die schwache Form hergeleitet:

$$\int_0^\ell \delta \epsilon \cdot E \epsilon A \, dx - \int_0^\ell \delta u \cdot n A \, dx - \delta u(\ell) \cdot F = 0$$

Ein Finites Element besteht in dem vorliegenden Beispiel aus 2 Knoten. Sowohl die Knoten, als auch die Elemente werden im Allgemeinen nummeriert, damit man sie eindeutig ansprechen kann. Im Bild sind die Elementnummern in den rechteckigen Kästen neben dem Element eingetragen. Die Knotennummern sind in den Kreisen neben den Knoten dargestellt. Die Knoten sind die Träger der primären Feldvariablen (hier: Verschiebung u) und Ziel der Finiten Elemente Berechnung ist die Bestimmung der primären Variablen, auch Freiheitsgrade (English: Degree of freedom **Dof**) an den Knoten.

The image shows a diagram of a bar element with two nodes. The element is represented by a horizontal line segment. The nodes are represented by circles at the ends of the segment. The element number is shown in a box next to the element, and the node numbers are shown in circles next to the nodes.

Auf diesen Elementen werden dann einfache Ansatzfunktionen verwendet, welche eine lineare Approximation des Verschiebungsfeldes darstellen:

$$N_1(\xi) = (1 - \xi) \quad N_2(\xi) = \xi \quad (1)$$

$$u_h(\xi) = N_1(\xi)\hat{u}_1^{(e)} + N_2(\xi)\hat{u}_2^{(e)} \quad (2)$$

$$u_h(\xi) = \sum_{I=1}^2 N_I(\xi)\hat{u}_I^{(e)} \quad (3)$$

```
[1]: import numpy as np
import plotly.graph_objects as go

# Define the x values
```

```

x = np.linspace(0, 1, 100)

N1 = 1-x
N2 = x

# Create the figure
fig = go.Figure()

# Add traces for each shape function
fig.add_trace(go.Scatter(x=x, y=N1, mode='lines', name='N_1'))
fig.add_trace(go.Scatter(x=x, y=N2, mode='lines', name='N_2'))

fig.update_layout(
    xaxis_title=r"$\xi$",
    yaxis_title=r'$N(\xi)$',
    legend=dict(x=1.05, y=1),
    font=dict(family="Serif", size=15),
    template='plotly_white',
    xaxis=dict(showgrid=True, gridcolor='grey', gridwidth=0.6, griddash='dash'),
    yaxis=dict(showgrid=True, gridcolor='grey', gridwidth=0.6, griddash='dash')
)

```

1.1 Die Steifigkeitsmatmatrix des Dehnstabes

Im nachfolgenden Python-Code wird die Steifigkeitsmatrix des Dehnstabes berechnet:

```

[2]: import sympy as sp
xi,ell,E,A = sp.symbols('xi, ell,E,A')

# Formfunktionen
N1 = 1-xi
N2 = xi
N = sp.Matrix([N1,N2])

# Ableitungen der Formfunktionen
dNdx1 = sp.diff(N,xi)

# Steifigkeitsmatrix
Kmat = sp.integrate(dNdx1*dNdx1.T,(xi,0,1))*E*A/ell
Kmat

```

```

[2]: 
$$\begin{bmatrix} \frac{AE}{\ell} & -\frac{AE}{\ell} \\ -\frac{AE}{\ell} & \frac{AE}{\ell} \end{bmatrix}$$


```

2 Analytische Lösung der DGL des Dehnstabes

Im folgenden wird die analytische Lösung der Differentialgleichung des Dehnstabes berechnet. Dafür wird die Differentialgleichung des Dehnstabes zweimal integriert. Die Integrationskonstanten

werden durch die Randbedingungen bestimmt.

```
[3]: E,A,x,n,F = sp.symbols('E,A,x,n,F')
u = sp.Function('u')
solution =sp.dsolve(
    E*A*u(x).diff(x,2)+n*A,u(x) # DGL
    ,ics={ # Randbedingungen
        u(0):0, # Verschiebungsrandbedingung
        u(x).diff(x).subs({"x":ell}):F/E/A # Kraftrandbedingung
    }
)
ufun=solution.rhs
ufun
```

$$[3]: -\frac{nx^2}{2E} + \frac{x(A\ell n + F)}{AE}$$

Die Schnittgröße $N(x)$ erhält man dann aus:

$$\sigma = E\epsilon = E\frac{\partial u}{\partial x}$$

$$N(x) = \sigma A = EA\frac{\partial u}{\partial x}$$

```
[4]: Nfun = E*A*solution.rhs.diff(x)
Nfun
```

$$[4]: AE\left(-\frac{nx}{E} + \frac{A\ell n + F}{AE}\right)$$

3 FEM Implementierung des Dehnstabs

Das folgende ist nur ein Ausdruck der FEM Implementierung aus dem Python Modul `StabFEM.py` und dient nur der Veranschaulichung:

```
[5]: from IPython.display import display, Markdown

# Open the Python file and read its content
with open('StabFEM.py', 'r') as file:
    content = file.read()

# Display the content as Markdown
display(Markdown(f'```\npython\n{content}\n```'))
```

```
import numpy as np
import matplotlib.pyplot as plt
```

```
class StabFEM:
    """
```

Klasse zur Durchführung von Finite-Elemente-Analysen für lineare Stabtragwerke im 2D-Raum.
"""

```
def __init__(self, numnp=2, numel=1):
    """
    Initialisiert eine neue Instanz der StabFEM-Klasse.

    Parameter:
    numnp (int): Anzahl der Knotenpunkte im Modell.
    numel (int): Anzahl der Elemente im Modell.
    """

    self.numnp = numnp
    self.numel = numel
    self.dim = 2
    self.nel = 2
    self.numdof = self.numnp * self.dim
    self.elements = np.zeros((self.numel, 2), dtype=int)
    self.eData = []
    self.coords = np.zeros((self.numnp, 2))
    self.dof = np.zeros((self.numnp, self.dim))
    self.eqind = np.zeros((self.numnp, self.dim), dtype=int)
    self.Kges = np.zeros((self.numdof, self.numdof))
    self.Fges = np.zeros((self.numnp, self.dim))
    self.dirichletBC = None
    self.Kuu = self.Kud = self.Kdu = self.Kdd = None

def resetFEM(self):
    self.dof = np.zeros((self.numnp, self.dim))
    self.eqind = np.zeros((self.numnp, self.dim), dtype=int)
    self.Kges = np.zeros((self.numdof, self.numdof))
    self.Fges = np.zeros((self.numnp, self.dim))

def setNodalCoordinates(self, X):
    """
    Setzt die Koordinaten der Knotenpunkte.

    Parameter:
    X (np.ndarray): 2D-Array mit den Koordinaten der Knotenpunkte.
    """
    self.coords[:, :] = X[:, :]

def setElementConnectivity(self, IX):
    """
    Definiert die Element-Knoten-Konnektivität.

    Parameter:
    IX (np.ndarray): Array mit den Indizes der Knoten für jedes Element.
    """
```

```

    for i in range(self.numel):
        for j in range(self.nel):
            self.elements[i, j] = int(IX[i, j])

def setElementData(self, areas, youngsmoduli, lineLoads):
    """
    Setzt die Elementdaten wie Querschnittsflächen, Elastizitätsmoduln und Längslasten.

    Parameter:
    areas (list): Liste der Querschnittsflächen für jedes Element.
    youngsmoduli (list): Liste der Elastizitätsmodul-Werte für jedes Element.
    lineLoads (list): Liste der Längslasten für jedes Element.
    """
    for i in range(self.numel):
        self.eData.append({
            "area": areas[i],
            "youngsmodulus": youngsmoduli[i],
            "lineLoad": lineLoads[i]
        })

def setDirichletBoundaryCondition(self, dirichletNodes, dirichletDir, dirichletVal):
    """
    Setzt die Dirichlet-Randbedingungen.

    Parameter:
    dirichletNodes (list): Liste der Knoten, für die die Dirichlet-Randbedingung gilt.
    dirichletDir (list): Liste der Richtungen (0:x, 1:y) der Randbedingungen.
    dirichletVal (list): Liste der Werte der Dirichlet-Randbedingungen.
    """
    eqID = 1
    for nodeID in range(self.numnp):
        for dirID in range(self.dim):
            self.eqind[nodeID, dirID] = eqID
            eqID += 1
    self.dirichletBC = np.zeros((len(dirichletNodes), 3))

    for i, (nodeID, dirId, dVal) in enumerate(zip(dirichletNodes, dirichletDir, dirichletVal)):
        self.eqind[nodeID, dirId] = -self.eqind[nodeID, dirId]
        self.dirichletBC[i, 0] = nodeID
        self.dirichletBC[i, 1] = dirId
        self.dirichletBC[i, 2] = dVal
        self.dof[nodeID, dirId] = dVal

def setExternalForces(self, neumannNodes, neumannDir, neumannVal):
    """
    Setzt die äußeren Kräfte auf die Knoten.

    Parameter:

```

```

    neumannNodes (list): Liste der Knoten, auf die Kräfte wirken.
    neumannDir (list): Liste der Richtungen (0:x, 1:y) der Kräfte.
    neumannVal (list): Liste der Kraftwerte für jeden Knoten.
    """
    for nodeID, nDir, nVal in zip(neumannNodes, neumannDir, neumannVal):
        self.Fges[nodeID, nDir] = nVal

def _getElementstiffness_(self, elID):
    """
    Berechnet die Steifigkeitsmatrix eines Elements.

    Parameter:
    elID (int): Die ID des Elements.

    Rückgabewert:
    np.ndarray: Steifigkeitsmatrix des Elements.
    """
    director = self._getElementDirector(elID)
    le = np.linalg.norm(director)
    Ke = self.eData[elID]["area"] * self.eData[elID]["youngsmodulus"] / le * np.ones((2, 2))
    Ke[0, 1] *= -1
    Ke[1, 0] *= -1
    return Ke

def assembleGlobalMatrix(self):
    """
    Assemblierung der globalen Steifigkeitsmatrix.
    """
    self._assembleGlobalMatrix2D()

def _assembleGlobalMatrix2D(self):
    """
    Assemblierung der globalen Steifigkeitsmatrix im 2D-Raum.
    """
    for elID in range(self.numel):
        Ke = self._getElementstiffness_(elID)
        Ke_enhanced = np.zeros((4, 4))
        Ke_enhanced[0, 0] = Ke[0, 0]
        Ke_enhanced[0, 2] = Ke[0, 1]
        Ke_enhanced[2, 0] = Ke[1, 0]
        Ke_enhanced[2, 2] = Ke[1, 1]
        R = self._getTransformationMatrix(elID)
        Ke = R.T @ Ke_enhanced @ R
        for nodeI in range(self.nel):
            globalNodeI = self.elements[elID, nodeI]
            for dimI in range(self.dim):
                rowID = int(np.sign(self.eqind[globalNodeI, dimI]) * self.eqind[globalNodeI, dimI])
                for nodeJ in range(self.nel):

```

```

        globalNodeJ = self.elements[elID, nodeJ]
        for dimJ in range(self.dim):
            colID = int(np.sign(self.eqind[globalNodeJ, dimJ]) * self.eqind[gl
            self.Kges[rowID, colID] += Ke[(nodeI * self.dim) + dimI, (nodeJ * s

def _getElementDirector(self, elID):
    """
    Bestimmt den Richtungsvektor eines Elements.

    Parameter:
    elID (int): Die ID des Elements.

    Rückgabewert:
    np.ndarray: Richtungsvektor des Elements.
    """
    nodeID1 = self.elements[elID, 0]
    nodeID2 = self.elements[elID, 1]
    director = self.coords[nodeID2, :] - self.coords[nodeID1, :]
    return director

def _getTransformationMatrix(self, elID):
    """
    Berechnet die Transformationsmatrix eines Elements.

    Parameter:
    elID (int): Die ID des Elements.

    Rückgabewert:
    np.ndarray: Transformationsmatrix des Elements.
    """
    director = self._getElementDirector(elID)
    le = np.linalg.norm(director)
    director = director / le
    normal = np.array([-director[1]], [director[0]])
    Rot = np.zeros((2, 2))
    E1 = np.zeros((2, 2))
    E1[0, 0] = 1.0
    E1[1, 1] = 1.0
    Euser = np.zeros((2, 2))
    Euser[0, 0] = director[0]
    Euser[1, 0] = director[1]
    Euser[0, 1] = normal[0]
    Euser[1, 1] = normal[1]
    Rot = Euser.T @ E1.T

    R = np.array([[Rot[0, 0], Rot[0, 1], 0, 0], [Rot[1, 0], Rot[1, 1], 0, 0], [0, 0, Rot[0
    return R

```

```

def assembleRightHandSide(self):
    """
    Assemblierung der rechten Seite des Gleichungssystems.
    """
    self._assembleRightHandSide2D()

def _assembleRightHandSide2D(self):
    """
    Assemblierung der rechten Seite im 2D-Raum.
    """
    for elID in range(self.numel):
        director = self._getElementDirector(elID)
        le = np.linalg.norm(director)
        R = self._getTransformationMatrix(elID)
        Feval = self.eData[elID]["area"] * le * self.eData[elID]["lineLoad"] * 0.5
        Fe = np.zeros((4, 1))
        Fe[0, 0] = 1
        Fe[2, 0] = 1
        Fe = Feval * (R.T @ Fe)
        for nodeI in range(self.nel):
            for dirI in range(self.dim):
                globalNodeI = self.elements[elID, nodeI]
                self.Fges[globalNodeI, dirI] += Fe[(nodeI * self.dim) + dirI]

def solveSystem(self):
    """
    Lösung des Gleichungssystems.
    """
    numcdof = self.dirichletBC.shape[0]
    numfdof = self.numdof - numcdof
    self.Kuu = np.zeros((numfdof, numfdof))
    self.Kud = np.zeros((numfdof, numcdof))
    self.Kdd = np.zeros((numcdof, numcdof))
    Fu = np.zeros((numfdof, 1))
    Fc = np.zeros((numcdof, 1))
    uc = np.zeros((numcdof, 1))
    eqf_inv = np.zeros((numfdof, 2), dtype=int)
    eqc_inv = np.zeros((numcdof, 2), dtype=int)
    icon = -1
    ifree = -1
    iIsFree = False
    for nodeI in range(self.numnp):
        for dirI in range(self.dim):
            jcon = -1
            jfree = -1
            jIsFree = False
            if self.eqind[nodeI, dirI] >= 0:

```



```

        iIsFree = True
        ifree += 1
    else:
        iIsFree = False
        icon += 1
    eqI = int(np.sign(self.eqind[nodeI, dirI]) * self.eqind[nodeI, dirI] - 1)
    if iIsFree:
        eqf_inv[ifree, 0] = nodeI
        eqf_inv[ifree, 1] = dirI
        Fu[ifree] = self.Fges[nodeI, dirI]
    else:
        eqc_inv[icon, 0] = nodeI
        eqc_inv[icon, 1] = dirI
        Fc[icon] = self.Fges[nodeI, dirI]
        uc[icon] = self.dof[nodeI, dirI]
    for nodeJ in range(self.numnp):
        for dirJ in range(self.dim):
            if self.eqind[nodeJ, dirJ] >= 0:
                jIsFree = True
                jfree += 1
            else:
                jIsFree = False
                jcon += 1
        eqJ = int(np.sign(self.eqind[nodeJ, dirJ]) * self.eqind[nodeJ, dirJ] - 1)
        if iIsFree and jIsFree:
            self.Kuu[ifree, jfree] = self.Kges[eqI, eqJ]
        if iIsFree and not jIsFree:
            self.Kud[ifree, jcon] = self.Kges[eqI, eqJ]
        if not iIsFree and not jIsFree:
            self.Kdd[icon, jcon] = self.Kges[eqI, eqJ]

    rhs = Fu - self.Kud @ uc
    u = np.linalg.solve(self.Kuu, rhs)
    ieq = 0
    for nodeI in range(numfdof):
        self.dof[eqf_inv[nodeI, 0], eqf_inv[nodeI, 1]] = u[ieq]
        ieq += 1
    rF = self.Kud.T @ u + self.Kdd @ uc - Fc
    ieq = 0
    for nodeI in range(numcdof):
        self.Fges[eqc_inv[nodeI, 0], eqc_inv[nodeI, 1]] = rF[ieq]
        ieq += 1

def computeNormalkraft(self, n=10):
    """
    Berechnet die Normalkräfte entlang der Stabstruktur.

    Args:

```

n (int): Anzahl der Stützpunkte je Element für die Berechnung.

Returns:

tuple: Koordinaten (np.ndarray) und Normalkräfte (np.ndarray) entlang der Stabstru

"""

```
X = np.zeros(n*self.numel)
```

```
Nges = np.zeros(n*self.numel)
```

```
for elID in range(self.numel):
```

```
    director = self._getElementDirector(elID)
```

```
    le = np.linalg.norm(director)
```

```
    node1 = self.elements[elID,0]
```

```
    node2 =self.elements[elID,1]
```

```
    x1 = self.coords[node1,0]
```

```
    x2 = self.coords[node2,0]
```

```
    X[elID*n:(elID+1)*n] = xlin = np.linspace(x1,x2,n)
```

```
    xilin = np.linspace(0,1,n)
```

```
    dofe = np.array([self.dof[node1,0],self.dof[node1,1],self.dof[node2,0],self.dof[node2,1]])
```

```
    R = self._getTransformationMatrix(elID)
```

```
    EA = self.eData[elID]["area"] * self.eData[elID]["youngsmodulus"]
```

```
    dof1 = R @ dofe.T
```

```
    for i,xi in enumerate(xilin):
```

```
        B=1.0/(le) * np.array([-1,0,1,0])
```

```
        N = EA * B @ dof1.T
```

```
        Nges[elID*n+i] = N
```

```
return X,Nges
```

```
def computeDisplacement(self,n=10):
```

```
    X = np.zeros(n*self.numel)
```

```
    uges = np.zeros(n*self.numel)
```

```
for elID in range(self.numel):
```

```
    director = self._getElementDirector(elID)
```

```
    le = np.linalg.norm(director)
```

```
    node1 = self.elements[elID,0]
```

```
    node2 =self.elements[elID,1]
```

```
    x1 = self.coords[node1,0]
```

```
    x2 = self.coords[node2,0]
```

```
    X[elID*n:(elID+1)*n] = xlin = np.linspace(x1,x2,n)
```

```
    xilin = np.linspace(0,1,n)
```

```
    dofe = np.array([self.dof[node1,0],self.dof[node1,1],self.dof[node2,0],self.dof[node2,1]])
```

```
    R = self._getTransformationMatrix(elID)
```

```
    R1 = R[0:2,0:2]
```

```
    dof1 = R @ dofe.T
```

```
    for i,xi in enumerate(xilin):
```

```
        N = np.array([1-xi,0,xi,0])
```

```
        u1 = N @ dof1.T
```

```
        uges[elID*n+i] = u1
```

```

    return X, uges

def getDisplacement(self, x):
    """
    Berechnet die Verschiebung an einer bestimmten Position x entlang der Struktur.

    Args:
        x (float): Position entlang der Struktur.

    Returns:
        float: Verschiebung an der Position x.
    """
    for elID in range(self.numel):
        node1 = self.elements[elID,0]
        node2 = self.elements[elID,1]
        x1 = self.coords[node1,0]
        x2 = self.coords[node2,0]

        if (x >= x1-1.e-12) and (x <= x2+1.e-12):
            director = self._getElementDirector(elID)
            le = np.linalg.norm(director)
            dofe = np.array([self.dof[node1,0], self.dof[node1,1], self.dof[node2,0], self.dof[node2,1]])
            R = self._getTransformationMatrix(elID)
            dofl = R @ dofe.T
            xi = (x-x1)/(x2-x1)
            N = np.array([1-xi, 0, xi, 0])
            R1 = R[0:2,0:2]
            ul = np.array([N @ dofl.T, 0])

            return R1.T @ ul.T

def plotMesh(self, deformed=False, scale=1.0, ax=None, fig=None):
    """
    Plottet das Netz des Modells.

    Parameter:
        deformed (bool): Gibt an, ob das deformierte Netz geplottet werden soll.
        scale (float): Skalierungsfaktor für die Darstellung.
        ax (matplotlib.axes.Axes): Achsenobjekt für die Darstellung.
        fig (matplotlib.figure.Figure): Figur-Objekt für die Darstellung.

    Rückgabewert:
        Tuple[matplotlib.figure.Figure, matplotlib.axes.Axes]: Figur und Achsenobjekt.
    """
    if ax is None:
        fig, ax = plt.subplots(1, 1)
    X = self.coords

```

```

Fnp = self.Fges
F = np.zeros((self.numnp, 2))
for n in range(self.numnp):
    for d in range(self.dim):
        F[n, d] = Fnp[n, d]

maxscale = float(np.max(np.max(X)))
minscale = float(np.min(np.min(X)))
ax.scatter(X[:, 0], X[:, 1], color="black", s=100)
for i in range(self.numel):
    x1 = X[self.elements[i, 0], :]
    x2 = X[self.elements[i, 1], :]
    ax.plot([x1[0], x2[0]], [x1[1], x2[1]], color="black", linewidth=5)

if deformed:
    u = self.dof
    Xd = np.zeros((self.numnp, 2))
    for dimI in range(self.dim):
        Xd[:, dimI] = X[:, dimI] + scale*u[:, dimI]
    ax.scatter(Xd[:, 0], Xd[:, 1], color="blue", s=100)
    for i in range(self.numel):
        x1 = Xd[self.elements[i, 0], :]
        x2 = Xd[self.elements[i, 1], :]
        ax.plot([x1[0], x2[0]], [x1[1], x2[1]], color="blue", linewidth=5, linestyle="solid")
    maxscale = float(np.max([maxscale, np.max(np.max(Xd))]))
    minscale = float(np.min([minscale, np.min(np.min(Xd))]))

ax.quiver(X[:, 0], X[:, 1], F[:, 0], F[:, 1], color="red")
ax.grid(True)
dx = (maxscale - minscale) * 0.1
ax.set_aspect('equal', adjustable='box')
ax.set_xlim(minscale - dx, maxscale + dx)
ax.set_ylim(minscale - dx, maxscale + dx)

return fig, ax
#  $K_{uu}u = F - K_{ud}u_d$ 
rhs = Fu - self.Kud @ uc
# display("RHS: ", rhs)
# display("Kuu:", self.Kuu)
u = np.linalg.solve(self.Kuu, rhs)
rhs = Fu - self.Kud @ uc
u = np.linalg.solve(self.Kuu, rhs)
# Kopiere zu globalen Freiheitsgraden
for nodeI in range(numfdof):
    # Kopiere zu globalen Freiheitsgraden
    ieq += 1
    self.dof[eqf_inv[nodeI, 0], eqf_inv[nodeI, 1]] = u[ieq]
    self.dof[eqf_inv[nodeI, 0], eqf_inv[nodeI, 1]] = u[ieq]

```

```

#  $R = K_{du} * u + K_{dd} * u_c - F_c$ 
# Bestimme Reaktionskräfte
#  $R = K_{du} * u + K_{dd} * u_c - F_c$ 
rF = self.Kud.T@u+self.Kdd@uc -Fc
# Kopiere zu den globalen Kräften
rF = self.Kud.T @ u + self.Kdd @ uc - Fc
self.Fges[eqc_inv[nodeI,0],eqc_inv[nodeI,1]] = rF[ieq]
ieq += 1
self.Fges[eqc_inv[nodeI,0],eqc_inv[nodeI,1]] = rF[ieq]
self.Fges[eqc_inv[nodeI, 0], eqc_inv[nodeI, 1]] = rF[ieq]

return fig,ax

```

4 Beispiele

4.1 Beispiel 1:

```

<div style="flex: 50%;">
  
</div>
<div style="flex: 50%;">
  <p>Gegebene Größen</p>
  <ul>
    <li>E = 210 GPa</li>
    <li>A = 33.4 cm2 (I-Profil - 200 )</li>
    <li>l = 4 m </li>
    <li>F = 5 kN</li>
    <li>n = 0.1 kN/m</li>
  </ul>
  <p>Gesucht</p>
  <ul>
    <li>u(x=4m)</li>
    <li>N(x=0m)</li>
    <li>Spannung an der Stelle x=0m</li>
  </ul>
</div>

```

```

[6]: from StabFEM import StabFEM
import numpy as np
import matplotlib.pyplot as plt

#####
# SetUp des Problems
#####

bsp1 = StabFEM(numnp=2,numel=1)
X = np.array([
    [0,0],

```

```

        [4000.0,0]
    ])
    elements = np.array([
        [0,1]
    ])
    areas = [33.4*10**2]
    youngsM = [210*10**3]
    loads = [0.1]

    # Setzen der Knotenkoordinaten, der Elementverbindungen und der Elementdaten
    bsp1.setNodalCoordinates(X)
    bsp1.setElementConnectivity(elements)
    bsp1.setElementData(areas,youngsM,loads)

    # Setzen der Randbedingungen
    bsp1.setDirichletBoundaryCondition([0,0,1],[0,1,1],[0,0,0]) # Knoten, Richtung,
    ↪Wert
    bsp1.setExternalForces([1],[0],[5000]) # Knoten, Richtung, Wert

    # Kontrolle der Eingaben
    print(f"Knotenkoordinaten:\n{bsp1.coords}")
    print(f"Elementverbindungen:\n{bsp1.elements}")
    print(f"Elementdaten:\n{bsp1.eData}")

```

Knotenkoordinaten:

```
[[ 0.  0.]
 [4000. 0.]]
```

Elementverbindungen:

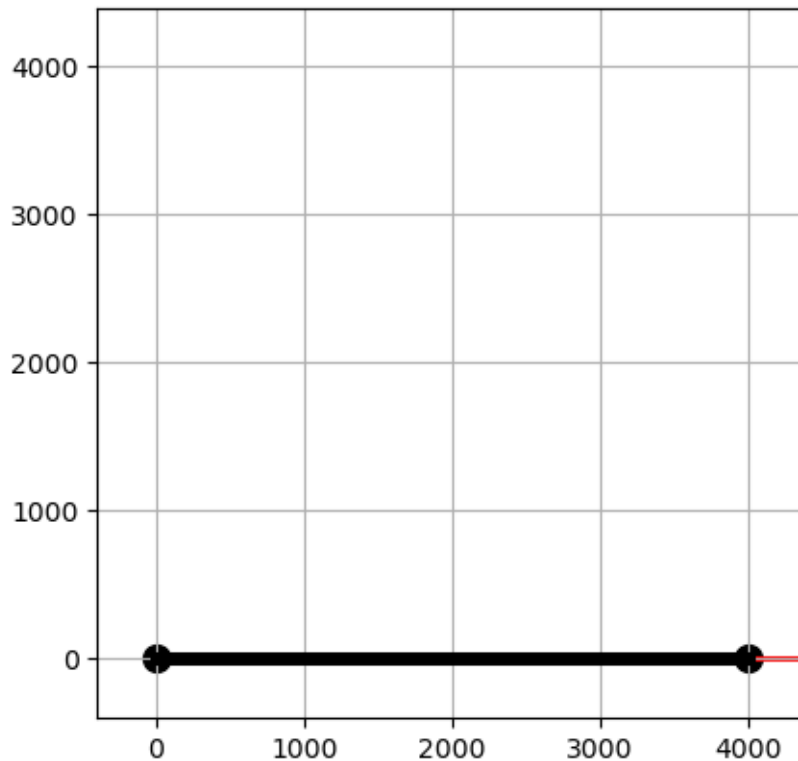
```
[[0 1]]
```

Elementdaten:

```
[{'area': 3340.0, 'youngsmodulus': 210000, 'lineLoad': 0.1}]
```

```
[7]: bsp1.plotMesh()
```

```
[7]: (<Figure size 640x480 with 1 Axes>, <Axes: >)
```



```
[8]: #####
# Lösen des Problems
#####

# Schleife über alle Elemente, bilden der Elementmatrizen und -vektoren und
↳assemblieren des globalen Systems
bsp1.assembleGlobalMatrix()
bsp1.assembleRightHandSide()
# Lösen des Gleichungssystems
bsp1.solveSystem()
display(r"$K_{ges}=$", (bsp1.Kges))
display("Displacement:", bsp1.dof)
display("Force:", bsp1.Fges)

'$K_{ges}=$'
array([[ 175350.,      0., -175350.,      0.],
       [      0.,      0.,      0.,      0.],
       [-175350.,      0.,  175350.,      0.],
       [      0.,      0.,      0.,      0.]])

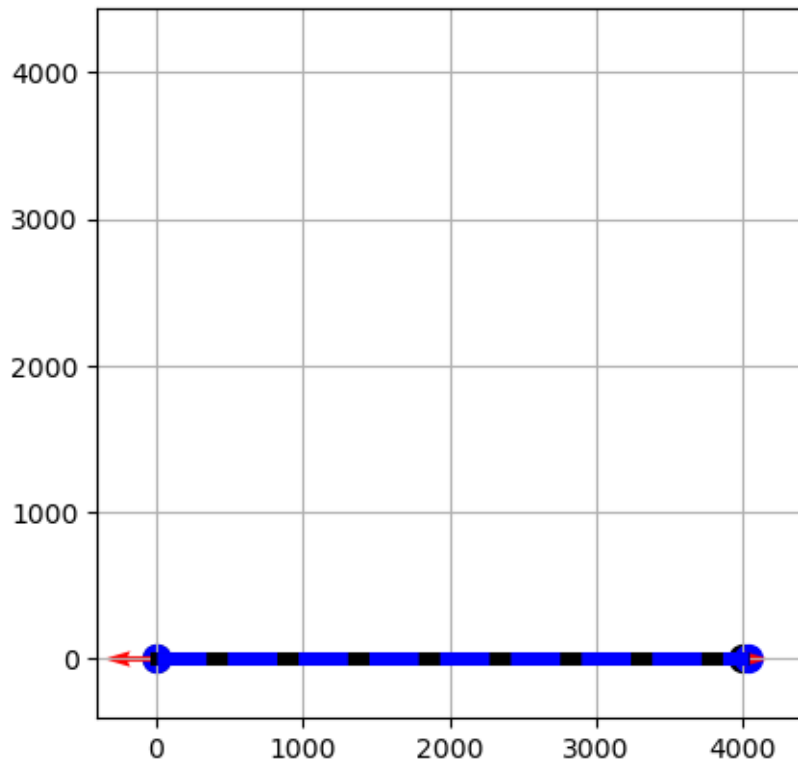
'Displacement:'
array([[0.,      0.],
```

```
[3.83803821, 0.      ]])
'Force:'
array([[ -1341000.,      0.],
       [  673000.,      0.]])
```

4.1.1 Darstellung der deformierten Struktur

```
[9]: bsp1.plotMesh(deformed = True, scale = 10)
```

```
[9]: (<Figure size 640x480 with 1 Axes>, <Axes: >)
```



4.1.2 Präsentation der Ergebnisse und Vergleich mit analytischer Lösung

Verschiebung am Ende des Balkens $u(x = \ell)$

```
[10]: print(f"Verschiebung am Ende des Balkens: {bsp1.getDisplacement(4000)} mm")
```

```
Verschiebung am Ende des Balkens: [3.83803821 0.      ] mm
```

Schnittkraft an der Einpannung $N(x = 0)$ Dies entspricht der Reaktionskraft:

```
[11]: print(f"Schnittkraft an der Einpannung: {-bsp1.Fges[0,0]/1000} kN")
```


Schnittkraft an der Einpannung: 1341.0 kN

4.2 Vergleich mit analytischer Lösung

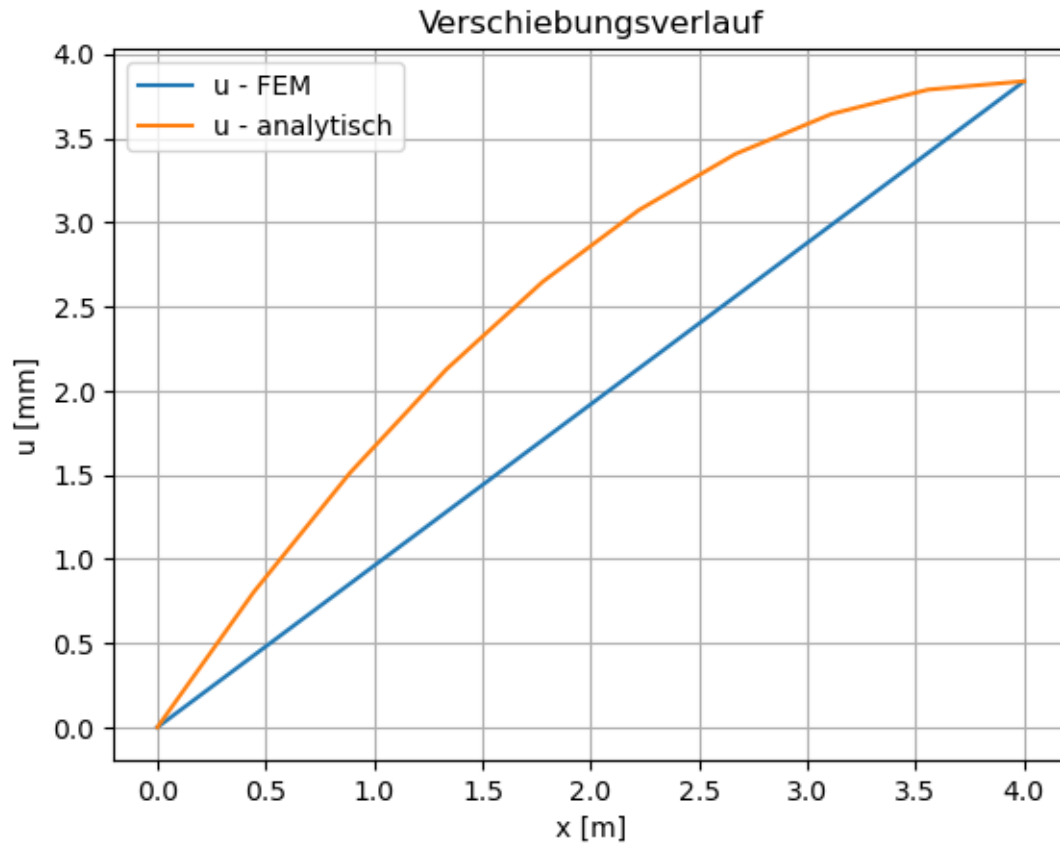
4.2.1 Verschiebungsverlauf $u(x)$

```
[12]: XN,u =bsp1.computeDisplacement()
fig,ax = plt.subplots(1,1)

ax.plot(XN/1000,u,label="u - FEM")
ax.set_title("Verschiebungsverlauf")
ax.set_xlabel("x [m]")
ax.set_ylabel("u [mm]")
ax.grid(True)

### Analytische Lösung
subs = {
    "E":bsp1.eData[0]["youngsmodulus"],
    "A":bsp1.eData[0]["area"],
    "n":bsp1.eData[0]["lineLoad"],
    "ell":4000,
    "F":5000
}
analytic_u = sp.lambdify(x,ufun.subs(subs),"numpy")
ax.plot(XN/1000,analytic_u(XN),label="u - analytisch")
ax.legend()
```

```
[12]: <matplotlib.legend.Legend at 0x7f1d208e5450>
```



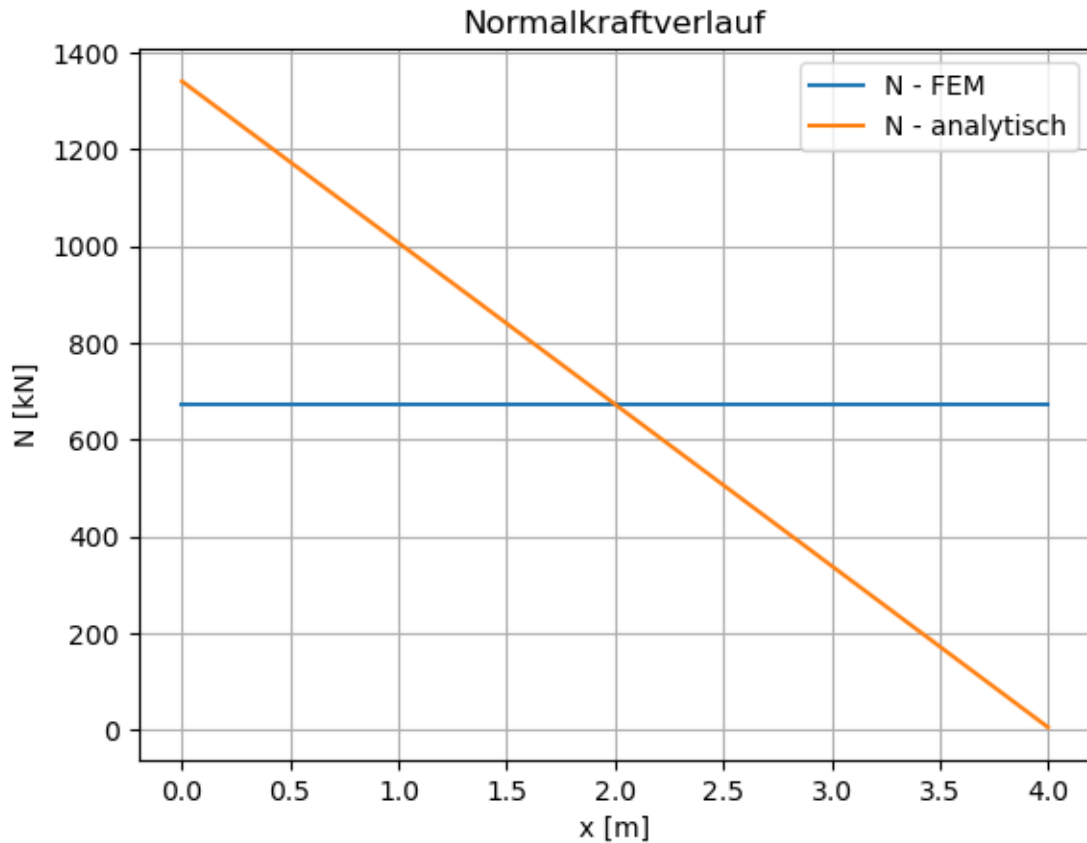
4.2.2 Normalkraftverlauf $N(x)$

```
[13]: XN,N =bsp1.computeNormalkraft()
fig,ax = plt.subplots(1,1)

ax.plot(XN/1000,N/1000,label="N - FEM")
ax.set_title("Normalkraftverlauf")
ax.set_xlabel("x [m]")
ax.set_ylabel("N [kN]")
ax.grid(True)

### Analytische Lösung
analytic_N = sp.lambdify(x,Nfun.subs(subs),"numpy")
ax.plot(XN/1000,analytic_N(XN)/1000,label="N - analytisch")
ax.legend()
```

```
[13]: <matplotlib.legend.Legend at 0x7f1d21d40bb0>
```



```
[14]: errorN = (N[0]-analytic_N(0))/(analytic_N(0))
print(f"Fehler bei x=0: {errorN*100:.2f}%")
```

Fehler bei x=0: -49.81%

4.2.3 Spannungsverlauf $\sigma(x)$

```
[15]: fig,ax = plt.subplots(1,1)

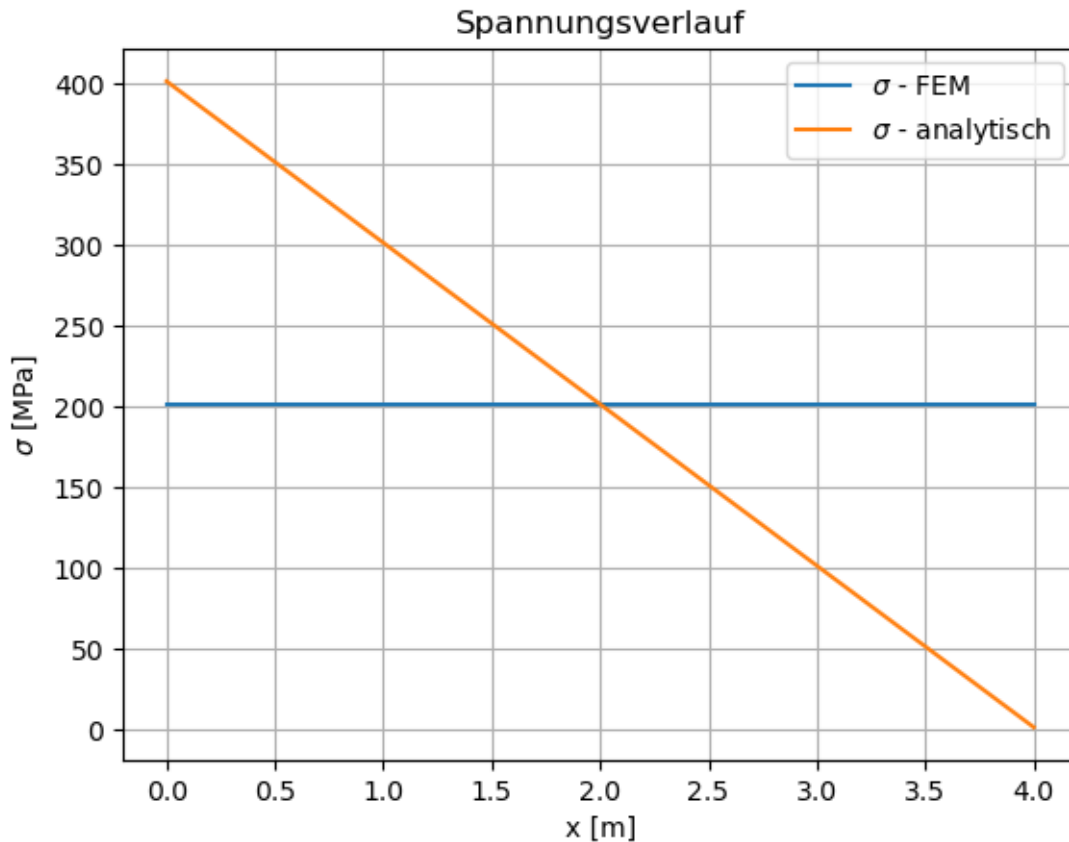
area = bsp1.eData[0]["area"]

ax.plot(XN/1000,N/area,label="$\\sigma$ - FEM")
ax.set_title("Spannungsverlauf")
ax.set_xlabel("x [m]")
ax.set_ylabel("$\\sigma$ [MPa]")
ax.grid(True)

### Analytische Lösung
analytic_N = sp.lambdify(x,Nfun.subs(subs),"numpy")
ax.plot(XN/1000,analytic_N(XN)/area,label="$\\sigma$ - analytisch")
```

```
ax.legend()
```

```
[15]: <matplotlib.legend.Legend at 0x7f1d204ed720>
```



5 Beispiel 2 - Fehleranalyse

- Wieviele Elemente benötigt man, damit der Fehler der Normalkraft kleiner als 5 % ist?
- Tragen Sie den Fehler in Abhängigkeit von der Anzahl der Elemente in einem Diagramm auf
- Welcher Zusammenhang besteht zwischen dem Fehler und der relativen Elementgröße (Elementlänge) $\frac{\ell_e}{\ell}$?

```
[16]: #####  
# Hilfe  
#####  
  
ell = 4000 # Länge des Stabes in mm  
numnp = 3 # Wieviele Knoten sollen erzeugt werden?  
  
# Definiere die Koordinaten der beiden Randknoten
```

```

X1 = np.array([0.0,0.0]) #mm
X2 = np.array([ell,0.0]) #mm

# Generate interpolation points
t = np.linspace(0, 1, numnp)
X = X1 + t[:, np.newaxis] * (X2 - X1)
print(f"Coordinates of the nodes: \n{X}")

# Generate Elements
IX = np.column_stack((np.arange(0, numnp-1), np.arange(1, numnp)))
print(f"Connectivity table: \n{IX}")

```

Coordinates of the nodes:

```

[[ 0.  0.]
 [2000. 0.]
 [4000. 0.]]

```

Connectivity table:

```

[[0 1]
 [1 2]]

```

[]:

6 Beispiel 3 - Stabsystem

Der dargestellte Verbundstab soll zwischen zwei feste Wände geklemmt werden. Für den Einbau wird das mittlere Stabteil mit der Kraft F zusammengedrückt. Wie groß muss F mindestens sein, damit der Einbau gelingt? Wie groß sind die Spannungen im Stab nach dem Einbau? Um wieviel ist das Mittelstück nach dem Einbau kürzer als vor dem Einbau?

```

<div style="flex: 50%;">
  
</div>
<div style="flex: 50%;">
  <p>Gegebene Größen</p>
  <ul>
    <li>E-Stahl = 210 GPa</li>
    <li>E-Cu = 105 GPa</li>
    <li>A-Stahl = 30 mm2</li>
    <li>A-Cu = 60 mm2</li>
    <li>a = 150 mm </li>
    <li>h = 3 mm </li>
  </ul>
  <p>Gesucht</p>
  <ul>
    <li>F um die Welle einzusetzen</li>
    <li>Spannungen im Stab nach dem Einbau</li>
    <li>Längenänderung des mittleren Wellenteils nach dem Einbau</li>
  </ul>
</div>

```

6.0.1 Welche Kraft ist notwendig um den Stab einzubauen?

```
[17]: #####
# SetUp des Problems
#####

a = 150
Astahl = 30
Acu = 60
EStahl = 210*10**3
ECu = 105*10**3
h = 3

bsp3 = StabFEM(numnp=4,numel=3)
X = np.array([
    [0,0],
    [a,0.0],
    [2*a,0],
    [3*a,0],
])
elements = np.array([
    [0,1],
    [1,2],
    [2,3]
])
areas = [Astahl,Acu,Astahl]
youngsM = [EStahl,ECu,EStahl]
loads = [0,0,0]

# Setzen der Knotenkoordinaten, der Elementverbindungen und der Elementdaten
bsp3.setNodalCoordinates(X)
bsp3.setElementConnectivity(elements)
bsp3.setElementData(areas,youngsM,loads)

# Setzen der Randbedingungen
bsp3.setDirichletBoundaryCondition([0,1,1,2,2,3],[1,0,1,0,1,1],[0,h,0,-h,0,0])
    ↪ # Knoten, Richtung, Wert
# bsp3.setExternalForces([1],[0],[5000]) # Knoten, Richtung, Wert

# Kontrolle der Eingaben
print(f"Knotenkoordinaten:\n{bsp3.coords}")
print(f"Elementverbindungen:\n{bsp3.elements}")
print(f"Elementdaten:\n{bsp3.eData}")
```

Knotenkoordinaten:

```
[ [ 0.  0.]
  [150. 0.]
  [300. 0.]
```

```

[450.  0.]]
Elementverbindungen:
[[0 1]
 [1 2]
 [2 3]]
Elementdaten:
[{'area': 30, 'youngsmodulus': 210000, 'lineLoad': 0}, {'area': 60,
'youngsmodulus': 105000, 'lineLoad': 0}, {'area': 30, 'youngsmodulus': 210000,
'lineLoad': 0}]

```

```
[18]: bsp3.plotMesh()
```

```
[18]: (<Figure size 640x480 with 1 Axes>, <Axes: >)
```

```

/home/steffen/anaconda3/envs/rise/lib/python3.10/site-
packages/matplotlib/quiver.py:695: RuntimeWarning:

```

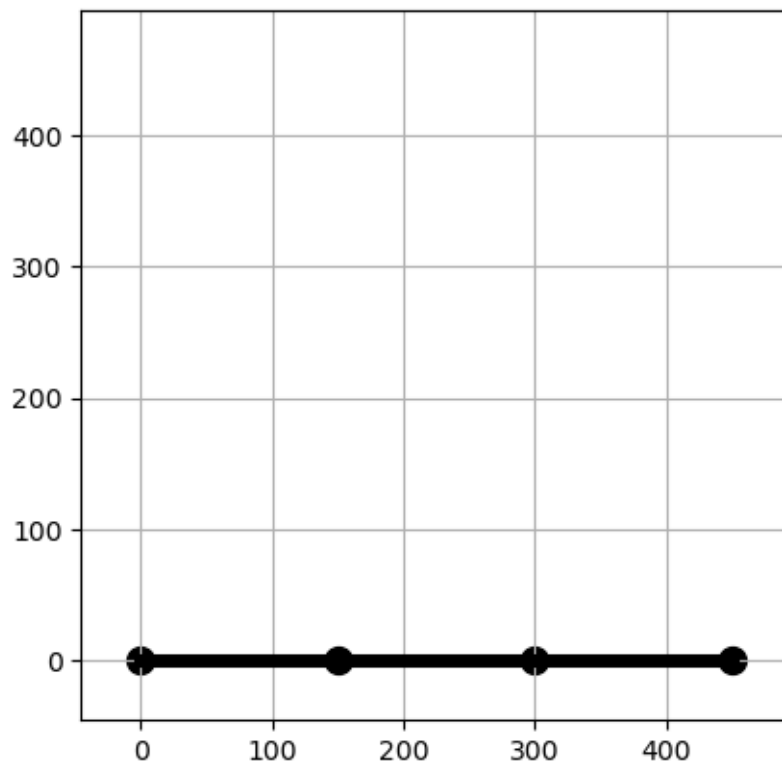
```
divide by zero encountered in scalar divide
```

```

/home/steffen/anaconda3/envs/rise/lib/python3.10/site-
packages/matplotlib/quiver.py:695: RuntimeWarning:

```

```
invalid value encountered in multiply
```



```
[19]: #####
# Lösen des Problems
#####

# Schleife über alle Elemente, bilden der Elementmatrizen und -vektoren und
↳assemblieren des globalen Systems
bsp3.assembleGlobalMatrix()
bsp3.assembleRightHandSide()
# Lösen des Gleichungssystems
bsp3.solveSystem()
display("Displacement:",bsp3.dof)
display("Force:",bsp3.Fges)
print(f'Kraft die notwendig ist, das mittlere Wellenstück zusammenzudrücken:↳
↳{bsp3.Fges[1,0]}')

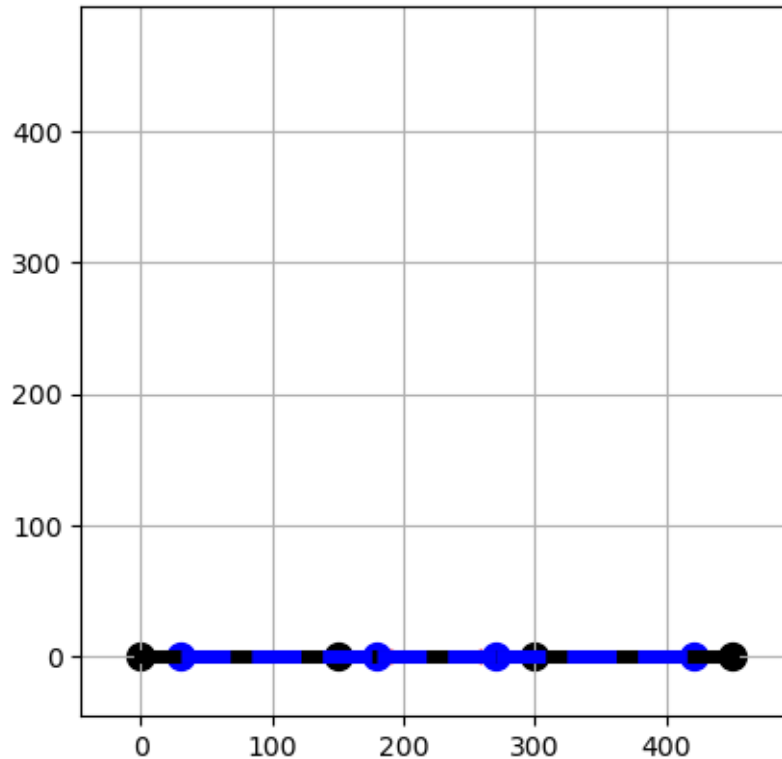
'Displacement:'
array([[ 3.,  0.],
       [ 3.,  0.],
       [-3.,  0.],
       [-3.,  0.]])

'Force:'
array([[ 0.,  0.],
       [252000.,  0.],
       [-252000.,  0.],
       [ 0.,  0.]])

Kraft die notwendig ist, das mittlere Wellenstück zusammenzudrücken: 252000.0
```

```
[20]: bsp3.plotMesh(deformed=True,scale=10)
```

```
[20]: (<Figure size 640x480 with 1 Axes>, <Axes: >)
```

6.0.2 Welche Spannung liegt nach dem Einbau vor

```
[21]: # Reset des Systems
bsp3.resetFEM()

# Setzen der Randbedingungen
bsp3.setDirichletBoundaryCondition([0,0,1,2,3,3],[0,1,1,1,0,1],[h/2,0,0,0,-h/
↪2,0]) # Knoten, Richtung, Wert

#####
# Lösen des Problems
#####

# Schleife über alle Elemente, bilden der Elementmatrizen und -vektoren und
↪assemblieren des globalen Systems
bsp3.assembleGlobalMatrix()
bsp3.assembleRightHandSide()
# Lösen des Gleichungssystems
bsp3.solveSystem()
display("Displacement:", bsp3.dof)
display("Force:", bsp3.Fges)
```

```

'Displacement:'
array([[ 1.5,  0. ],
       [ 0.5,  0. ],
       [-0.5,  0. ],
       [-1.5,  0. ]])

'Force:'
array([[ 42000.,  0.],
       [  0.,  0.],
       [  0.,  0.],
       [-42000.,  0.]])

```

```

[22]: XN,N =bsp3.computeNormalkraft()
fig,ax = plt.subplots(1,1)

ax.plot(XN/1000,N/1000,label="N - FEM")
ax.set_title("Normalkraftverlauf")
ax.set_xlabel("x [m]")
ax.set_ylabel("N [kN]")
ax.grid(True)

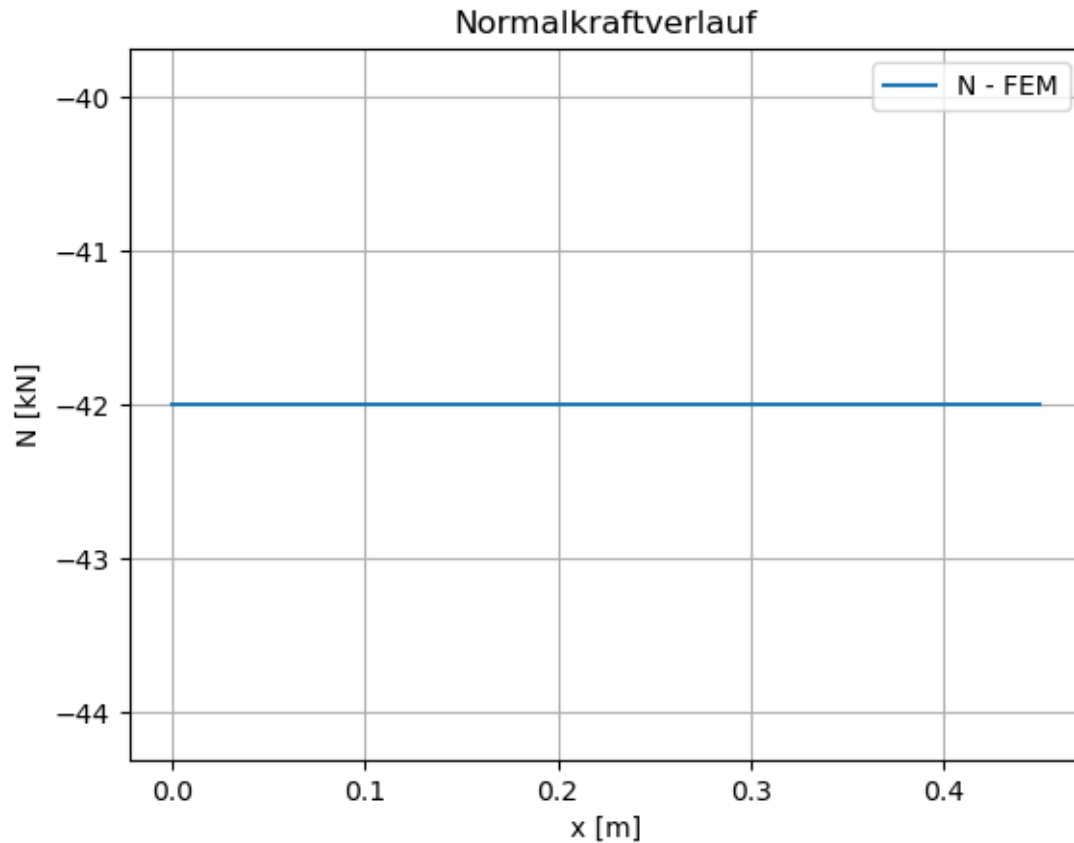
ax.legend()
sigstahl = N[0]/(bsp3.eData[0]['area'])
sigcu    = N[0]/(bsp3.eData[1]['area'])
print(f'Spannungen im Stahlstab: {sigstahl}')
print(f'Spannungen im Kupferstab: {sigcu}')

```

```

Spannungen im Stahlstab: -1400.0
Spannungen im Kupferstab: -700.0

```



6.0.3 Dehnung im mittleren Stabteil

$$\epsilon = \frac{u_2 - u_1}{a}$$

```
[23]: eps = (bsp3.dof[2,0]-bsp3.dof[1,0])/(a)
print(f'Dehnung im mittleren Stab: {eps}')
```

Dehnung im mittleren Stab: -0.006666666666666667

7 Beispiel 4

```
<div style="flex: 50%;">
  
</div>
<div style="flex: 50%;">
  <p>Gegebene Größen</p>
  <ul>
    <li>E-Stahl = 210 GPa</li>
    <li>A = 33.4 cm² (I-Profil - 200 )</li>
    <li>a = 5 m </li>
  </ul>
</div>
```

```

        <li>F = 10 kN </li>
    </ul>
    <p>Gesucht</p>
    <ul>
        <li>Verschiebung im Kraftangriffspunkt</li>
    </ul>
</div>

```

```

[24]: #####
# SetUp des Problems
#####

a = 5*1000
A = 33.4*10**2
E = 210*10**3
F = -1000*10**3

X = np.array([
    [0,0],
    [1.5*a,0.0],
    [2*1.5*a,0],
    [0.5*1.5*a,a],
    [0.5*1.5*a+1.5*a,a],
])
elements = np.array([
    [0,1],
    [1,2],
    [0,3],
    [3,1],
    [1,4],
    [4,2],
    [3,4]
])

numnp = X.shape[0]
numel = elements.shape[0]

bsp4 = StabFEM(numnp,numel)

areas = [A for i in range(0,numel)]
youngsM = [E for i in range(0,numel)]
loads = [0 for i in range(0,numel)]

# Setzen der Knotenkoordinaten, der Elementverbindungen und der Elementdaten
bsp4.setNodalCoordinates(X)
bsp4.setElementConnectivity(elements)
bsp4.setElementData(areas,youngsM,loads)

```

```

# Setzen der Randbedingungen
bsp4.setDirichletBoundaryCondition([0,0,2,2],[0,1,0,1],[0,0,0,0]) # Knoten,
    ↳Richtung, Wert
bsp4.setExternalForces([1],[1],[F])

# Kontrolle der Eingaben
print(f"Knotenkoordinaten:\n{bsp4.coords}")
print(f"Elementverbindungen:\n{bsp4.elements}")
print(f"Elementdaten:\n{bsp4.eData}")

```

Knotenkoordinaten:

```

[[ 0.  0.]
 [7500. 0.]
 [15000. 0.]
 [ 3750. 5000.]
 [11250. 5000.]]

```

Elementverbindungen:

```

[[0 1]
 [1 2]
 [0 3]
 [3 1]
 [1 4]
 [4 2]
 [3 4]]

```

Elementdaten:

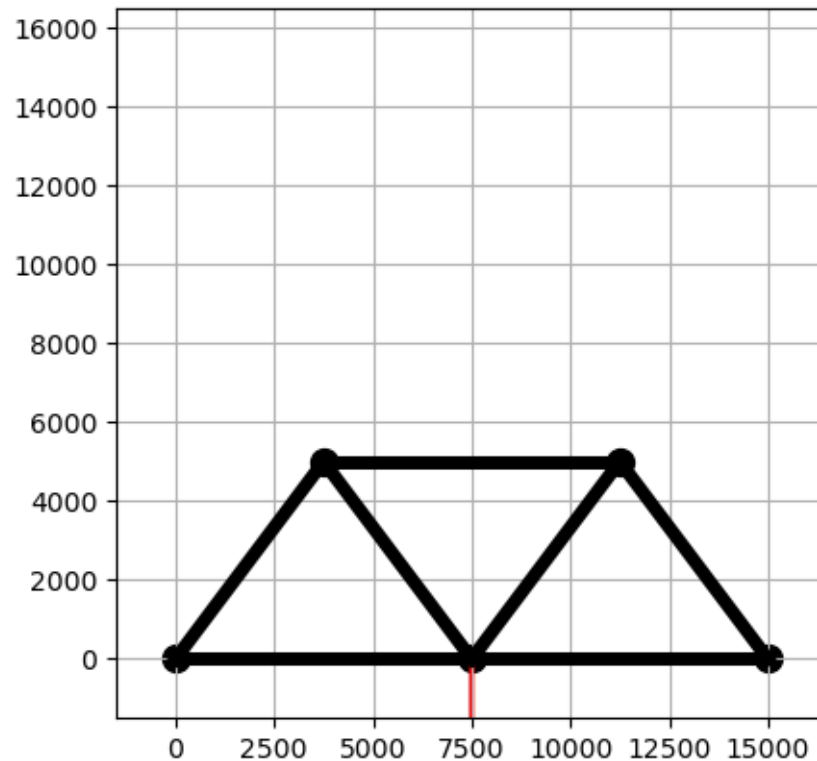
```

[{'area': 3340.0, 'youngsmodulus': 210000, 'lineLoad': 0}, {'area': 3340.0,
'youngsmodulus': 210000, 'lineLoad': 0}, {'area': 3340.0, 'youngsmodulus':
210000, 'lineLoad': 0}, {'area': 3340.0, 'youngsmodulus': 210000, 'lineLoad':
0}, {'area': 3340.0, 'youngsmodulus': 210000, 'lineLoad': 0}, {'area': 3340.0,
'youngsmodulus': 210000, 'lineLoad': 0}, {'area': 3340.0, 'youngsmodulus':
210000, 'lineLoad': 0}]

```

```
[25]: bsp4.plotMesh()
```

```
[25]: (<Figure size 640x480 with 1 Axes>, <Axes: >)
```



```
[26]: #####
# Lösen des Problems
#####

# Schleife über alle Elemente, bilden der Elementmatrizen und -vektoren und
↳assemblieren des globalen Systems
bsp4.assembleGlobalMatrix()
bsp4.assembleRightHandSide()
# Lösen des Gleichungssystems
bsp4.solveSystem()
print(f'Verschiebung im Kraftangriffspunkt: {bsp4.dof[1,1]} mm')
```

Verschiebung im Kraftangriffspunkt: -19.93780296549757 mm

```
[27]: bsp4.plotMesh(deformed=True,scale=100)
```

```
[27]: (<Figure size 640x480 with 1 Axes>, <Axes: >)
```

